

P0288R4

any_invocable

Published Proposal, 2019-06-16

This version:

<http://wg21.link/p0288r4>

Authors:

Ryan McDougall <mcdougall.ryan@gmail.com>

Matt Calabrese <metaprogrammingtheworld@gmail.com>

Audience:

LWG, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a conservative, move-only equivalent of `std::function`.

Table of Contents

1	Brief History
2	Overview
3	Specification
4	Polls from LEWG San Diego Review (2018)
4.1	Support <code>func()</code> , <code>func() const</code> , <code>func() &&</code>
4.2	Support <code>func() &&</code> only
4.3	Remove <code>target/target_type</code>
4.4	Require more stuff (<code>noexcept</code> , <code>const&&</code> , ...)
4.5	Name Options
5	Polls from LEWG Kona Review (2019)
5.1	We want to spend time on this now in order to include it in C++20
5.2	Add support for <code>func() const&</code> and <code>func()&</code>
5.3	Add support for <code>func() noexcept</code> (x all of the above)
5.4	Include the option for CTAD
5.5	Name: <code>callable</code> vs <code>any invocable</code>
5.6	<code>any_invocable</code> vs <code>invocable</code>
5.7	Header choice
5.8	Can get <code>std::function</code> from <code><any_invocable></code>
5.9	Can get <code>std::function</code> from <code><invocable></code>
5.10	Remove the null-check in the call operator and throwing of <code>bad_function_call</code>
5.11	Remove the null-check in constructors that are not <code>nullptr_t</code>
5.12	Perfect forwarding for converting constructor instead of by-value
5.13	Forward to LWG for C++20
6	Implementation Experience
7	References

§ 1. Brief History

This paper started as a proposal by David Krauss, N4543[1], from 2015 and there has been an open issue in the LEWG bugzilla requesting such a facility since 2014[2].

Since then, the paper has gone through 4 revisions and has been considered in small groups in LEWG multiple times. Gradual feedback has led to the conservative proposal seen here. The most-recent draft prior to this was a late-paper written and presented by Ryan McDougall in LEWGI in San Diego[3]. It included multiple references to implementations of move-only functions and made a strong case for the importance of a move-only form of `std::function`.

Feedback given was encouragement for targeting C++20.

An updated version of that paper was presented on Saturday at the end of the San Diego meeting. Poll results from that presentation are presented after the overview in this document.

The revision was presented in Kona, receiving additional polls and feedback, and was to be forwarded to LWG pending updates reflecting additional poll results. Those changes have been applied to the wording in this paper. Polls from the LEWG Kona review are also provided in this document.

§ 2. Overview

This conservative `any_invocable` is intended to be the same as `std::function`, with the exceptions of the following:

1. It is move-only.
2. It does not have the const-correctness bug of `std::function` detailed in n4348.[4]
3. It provides support for cv/ref/noexcept qualified function types.
4. It does not have the `target_type` and `target` accessors (direction requested by users and implementors).
5. Invocation has strong preconditions.

§ 3. Specification

The following is relative to N4810.[5]

Add `<any_invocable>` to Table 19 — C++ library headers

```
Header <any_invocable> synopsis [inv.syn]

namespace std {
    template<class Sig> class any_invocable; // not defined

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...)> { see below };

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) &&> { see below };

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) &&> { see below };

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) const> { see below };

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) const &&> { see below };

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) noexcept> { see below };
```

```

template<class R, class... ArgTypes>
    class any_invocable<R(ArgTypes...) & noexcept> { see below };

template<class R, class... ArgTypes>
    class any_invocable<R(ArgTypes...) && noexcept> { see below };

template<class R, class... ArgTypes>
    class any_invocable<R(ArgTypes...) const noexcept> { see below };

template<class R, class... ArgTypes>
    class any_invocable<R(ArgTypes...) const & noexcept> { see below };
}

```

- 1 The class definitions for the partial specializations of `any_invocable` mentioned above are specified using exposition-only macros `QUAL_OPT`, `QUAL_OPT_REF`, and `NOEXCEPT_VAL`, as described below.
- 2 For each partial specialization of `any_invocable` mentioned above, let `QUAL_OPT` be an exposition-only macro defined to be a textual representation of the cv and reference qualifiers of the function type parameter of `any_invocable`.
- 3 *[Note:*
 For `any_invocable<void() const>`
 - `QUAL_OPT` is `const`

 For `any_invocable<void() &&>`
 - `QUAL_OPT` is `&&`

 For `any_invocable<void()>`
 - `QUAL_OPT` is
]
- 4 For each partial specialization of `any_invocable` mentioned above, let `NOEXCEPT_VAL` be an exposition-only macro defined to be true if the function type parameter of `any_invocable` is `noexcept`, otherwise false.
- 5 *[Note:*
 For `any_invocable<void() noexcept>`
 - `NOEXCEPT_VAL` is `true`

 For `any_invocable<void()>`
 - `NOEXCEPT_VAL` is `false`
]
- 6 For each partial specialization of `any_invocable` mentioned above, let `QUAL_OPT_REF` be an exposition-only macro defined in the following manner:
 - If the function type parameter of `any_invocable` is reference qualified, let `QUAL_OPT_REF` be defined as `QUAL_OPT`.
 - Otherwise, let `QUAL_OPT_REF` be defined as `QUAL_OPT&`.
- 7 *[Note:*
 For `any_invocable<void() const>`
 - `QUAL_OPT_REF` is `const &`

 For `any_invocable<void() &&>`
 - `QUAL_OPT_REF` is `&&`

 For `any_invocable<void()>`
 - `QUAL_OPT_REF` is `&`
]
- 8 For each partial specialization of `any_invocable` mentioned above, the following is a synopsis of that partial specialization and its associated non-member functions.

```

namespace std {

```

```

template<class R, class... ArgTypes>
class any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)> {
public:
    using result_type = R;

    // 20.14.16.2.1, construct/copy/destroy
    any_invocable() noexcept;
    any_invocable(nullptr_t) noexcept;
    any_invocable(any_invocable&&) noexcept;
    template<class F> any_invocable(F&&);

    template<class T, class... Args>
        explicit any_invocable(in_place_type_t<T>, Args&&...);
    template<class T, class U, class... Args>
        explicit any_invocable(in_place_type_t<T>, initializer_list<U>, Args&&...);

    any_invocable& operator=(any_invocable&&) noexcept;
    any_invocable& operator=(nullptr_t) noexcept;
    template<class F> any_invocable& operator=(F&&);
    template<class F> any_invocable& operator=(reference_wrapper<F>) noexcept;

    ~any_invocable();

    // 20.14.16.2.2, any_invocable modifiers
    void swap(any_invocable&) noexcept;

    // 20.14.16.2.3, any_invocable capacity
    explicit operator bool() const noexcept;

    // 20.14.16.2.4, any_invocable invocation
    R operator()(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL);
};

/ 20.14.16.2.6, Null pointer comparisons
template<class R, class... ArgTypes>
    bool operator==(const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&, nullptr_t) noexcept;

template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&) noexcept;

template<class R, class... ArgTypes>
    bool operator!=(const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&, nullptr_t) noexcept;

template<class R, class... ArgTypes>
    bool operator!=(nullptr_t, const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&) noexcept;

// 20.14.16.2.7, specialized algorithms
template<class R, class... ArgTypes>
    void swap(any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&,
              any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>&) noexcept;
}

```

9 The `any_invocable` class template provides polymorphic wrappers that generalize the notion of a callable object (20.14.2). These wrappers can store, move, and call arbitrary callable objects (20.14.2), given a call signature (20.14.2), allowing functions to be first-class objects.

10 A callable type (20.14.2) `F` is `Callable` for argument types `ArgTypes` and return type `R` if the expression
`INVOKE<R>(declval<F>(), declval<ArgTypes>()...),`
 considered as an unevaluated operand (7.2), is well-formed (20.14.3).

any_invocable() noexcept;

1 *Ensures*: !*this.

any_invocable(nullptr_t) noexcept;

2 *Ensures*: !*this.

any_invocable(any_invocable&& f) noexcept;

3 *Ensures*: If !f, *this has no target; otherwise, the target of *this is equivalent to the target of f before the construction, and f is in a valid state with an unspecified value.

4 [Note: Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, wheref's target is an object holding only a pointer or reference to an object and a member function pointer.— end note]

template<class F> any_invocable(F&& f);

5 *Requires*: decay<F> shall be Cpp17MoveConstructible.

6 *Constraints*:

(6.1) — decay_t<F> is not the same type as any_invocable and

(6.2) — decay_t<F> is not a specialization of in_place_type_t and

(6.3) — is_constructible_v<decay_t<F>, F> is true and

(6.4) — is_move_constructible_v<decay_t<F>> is true and

(6.5) — F QUAL_OPT_REF is Callable (20.14.16.2) for argument types ArgTypes..., return type R, and

(6.6) — !NOEXCEPT_VAL || noexcept(INVOKE<R>(declval<F QUAL_OPT_REF>(), declval<ArgTypes>()...)) is true.

7 *Ensures*: !*this if any of the following hold:

—(7.1) f is a null function pointer value.

—(7.2) f is a null member pointer value.

—(7.3) F is an instance of the any_invocable class template, and !f.

8 Otherwise, *this targets a copy of f initialized with std::forward<F>(f). [Note: Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where f is an object holding only a pointer or reference to an object and a member function pointer.— end note]

9 *Throws*: Shall not throw exceptions when f is a function pointer or a reference_wrapper<T> for some T. Otherwise, may throw bad_alloc or any exception thrown by the expression decay_t<F>(std::forward<F>(f)).

template<class T, class... Args>
explicit any_invocable(in_place_type_t<T>, Args&&... args);

10 Let VT be decay_t<T>.

11 *Requires*: VT shall be Cpp17MoveConstructible.

12 *Constraints*:

(12.1) — is_constructible_v<VT, Args...> is true and

(12.2) — is_move_constructible_v<VT> is true and

(12.3) – VT QUAL_OPT_REF is Callable (20.14.16.2) for argument types
ArgTypes..., return type R, and

(12.4) – !NOEXCEPT_VAL || noexcept(INVOKE<R>(declval<F QUAL_OPT_REF>(), declval<ArgTypes>()...)) is true.

13 *Ensures*: *this targets an object of type VT initialized
with std::forward<Args>(args)... by *direct-non-list-initialization*. [Note:
Implementations should avoid the use of dynamically allocated memory for small
callable objects, for example, where f is an object holding only a pointer or
reference to an object and a member function pointer.– end note]

```
template<class T, class U, class... Args>  
    explicit any_invocable(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);
```

14 Let VT be decay_t<T>.

15 *Requires*: VT shall be Cpp17MoveConstructible.

16 *Constraints*:

(16.1) – is_constructible_v<VT, initializer_list<U>&, Args...> is true and

(16.2) – is_move_constructible_v<VT> is true and

(16.3) – VT QUAL_OPT_REF is Callable (20.14.16.2) for argument types
ArgTypes..., return type R, and

(16.4) – !NOEXCEPT_VAL || noexcept(INVOKE<R>(declval<F QUAL_OPT_REF>(), declval<ArgTypes>()...)) is true.

17 *Ensures*: *this targets an object of type VT initialized
with ilist, std::forward<ArgTypes>(args)... by *direct-non-list-initialization*. [Note:
Implementations should avoid the use of dynamically allocated memory for small
callable objects, for example, where f is an object holding only a pointer or
reference to an object and a member function pointer.– end note]

```
any_invocable& operator=(any_invocable&& f) noexcept;
```

18 *Effects*: Replaces the target of *this with the target of f.

19 *Returns*: *this.

```
any_invocable& operator=(nullptr_t) noexcept;
```

20 *Effects*: If *this != nullptr, destroys the target of this.

21 *Ensures*: !(*this).

22 *Returns*: *this.

```
template<class F> any_invocable& operator=(F&& f);
```

23 *Effects*: As if by: any_invocable(std::forward<F>(f)).swap(*this);

24 *Returns*: *this.

25 *Requires*: decay<F> shall be Cpp17MoveConstructible.

26 *Constraints*:

(26.1) – decay_t<F> is not the same type as any_invocable and

(26.3) – is_constructible_v<decay_t<F>, F> is true and

(26.4) – is_move_constructible_v<decay_t<F>> is true and

(26.5) – F QUAL_OPT_REF is Callable (20.14.16.2) for argument types

```

ArgTypes..., return type R, and

(26.4) — !NOEXCEPT_VAL || noexcept(INVOKE<R>(declval<F QUAL_OPT_REF>(), declval<ArgTypes>()...)) is true.

template<class F> any_invocable& operator=(reference_wrapper<F> f) noexcept;

27 Effects: As if by: any_invocable(f).swap(*this);

28 Returns: *this.

~any_invocable();

29 Effects: If *this != nullptr, destroys the target of this.

SECTION.2 Modifiers [inv.wrap.func.mod]
void swap(any_invocable& other) noexcept;

1 Effects: Interchanges the targets of *this and other.

SECTION.3 Capacity [inv.wrap.func.cap]
explicit operator bool() const noexcept;

1 Returns: true if *this has a target, otherwise false.

SECTION.4 Invocation [inv.wrap.func.inv]
R operator()(ArgTypes... args) QUAL_OPT noexcept(NOEXCEPT_VAL);

1 Returns: INVOKE<R>(static_cast<decltype(f) QUAL_OPT_REF>(f),
    std::forward<ArgTypes>(args)...) , where f is the target object of *this,.

2 Requires: (bool)*this is true.

3 Throws: Any exception thrown by the wrapped callable object.

SECTION.5 Null pointer comparison functions [inv.wrap.func.nullptr]
template<class R, class... ArgTypes>
    bool operator==(const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f) noexcept;

1 Returns: !f.

template<class R, class... ArgTypes>
    bool operator!=(const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator!=(nullptr_t, const any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f) noexcept;

2 Returns: (bool)f.

SECTION.6 Specialized algorithms [inv.wrap.func.alg]
template<class R, class... ArgTypes>
    void swap(any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f1,
        any_invocable<R(ArgTypes...) QUAL_OPT noexcept(NOEXCEPT_VAL)>& f2) noexcept;

1 Effects: As if by: f1.swap(f2)

```

§ 4. Polls from LEWG San Diego Review (2018)

§ 4.1. Support func(), func() const, func() &&

```
SF F N A SA
6 6 2 1 0
```

§ 4.2. Support func() && only

```
SF F N A SA
2 2 7 1 1
```

§ 4.3. Remove target/target_type

```
SF F N A SA
12 5 0 0 0
```

§ 4.4. Require more stuff (noexcept, const&&, ...)

```
SF F N A SA
0 1 8 6 0
```

Note that the final poll (require more stuff) was not due to members being against the design, but because we could easily add those facilities in a later standard without any breakage.

§ 4.5. Name Options

There was one final poll, which brought us to the name `any_invocable`.

```
3 unique_function
3 move_function
2 move_only_function
7 movable_function
8 mfunction
10 any_invocable
8 mofun
8 mofunc
0 callback
4 mvfunction
2 func
0 callable
2 any_function
```

§ 5. Polls from LEWG Kona Review (2019)

§ 5.1. We want to spend time on this now in order to include it in C++20

```
SF F N A SA
8 8 2 0 0
```

§ 5.2. Add support for func() const& and func()&

```
SF F N A SA
0 8 7 0 0
```


5.3. Add support for func() noexcept (x all of the above)

```
SF F N A SA
2 12 2 0 0
```

5.4. Include the option for CTAD

```
SF F N A SA
0 1 5 9 0
```

5.5. Name: callable vs any invocable

```
SC C N AI SAI
3 2 3 5 6
```

5.6. any_invocable vs invocable

```
SAI AI N I SI
3 7 2 5 1
```

5.7. Header choice

```
7 <functional>
11 <any_invocable>
11 <invocable>
3 <()>
```

5.8. Can get std::function from <any_invocable>

```
SF F N A SA
0 1 4 4 7
```

5.9. Can get std::function from <invocable>

```
SF F N A SA
1 3 6 3 2
```

Decide on <any_invocable>. Unanimous for <functional> to pull it in, even if in its own header.

5.10. Remove the null-check in the call operator and throwing of bad_function_call

```
SF F N A SA
8 2 1 0 0
```

5.11. Remove the null-check in constructors that are not nullptr_t

```
std::any_callable ac = my_ptr_object;  
if(ac) { /* true even if my_ptr is nullptr */ }
```

```
SF F N A SA  
0 2 2 4 3
```

§ 5.12. Perfect forwarding for converting constructor instead of by-value

Unanimous

§ 5.13. Forward to LWG for C++20

```
SF F N A SA  
8 5 0 0 0
```

§ 6. Implementation Experience

There are many implementations of a move-only `std::function` with a design that is similar to this. What is presented is a conservative subset of those implementations. The changes suggested in LEWG, though minimal, have not been used in a large codebase.

Previous revisions of this paper have included publicly accessible move-only function implementations, notably including implementations in HPX, Folly, and LLVM.

§ 7. References

[1]: David Krauss: N4543 "A polymorphic wrapper for all Callable objects" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4543.pdf>

[2]: Geoffrey Romer: "Bug 34 - Need type-erased wrappers for move-only callable objects" https://issues.isocpp.org/show_bug.cgi?id=34

[3]: Ryan McDougall: P0288R2 "The Need for `std::unique_function`" <https://wg21.link/p0288r2>

[4]: Geoffrey Romer: N4348 "Making `std::function` safe for concurrency" www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4348.html

[5]: Richard Smith: N4778 "Working Draft, Standard for Programming Language C++" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4810.pdf>